FHQ Treap（无旋Treap）

一、FHQ Treap简介

1.1 什么是FHQ Treap?

FHQ Treap（无旋Treap）是由OI选手范浩强发明的一种基于**分裂**和**合并**操作的Treap变种，它不需要旋转操作就能实现所有平衡树功能。

1.2 核心特点

- **无旋转**：通过分裂(split)和合并(merge)操作替代旋转
- **函数式**：操作不修改原树，返回新树，天然支持可持久化
- **强大**：能用于区间翻转、移动、持久化

旋转法和FHQ维护的都是Treap树，Treap树的最后形态由键值和优先级确定。旋转法和FHQ的区别是维护的方法不同，结果是一样的。

二、数据结构定义

2.1 节点结构

```
struct Node{
        int ls,rs;          //左儿子lson，右儿子rson
        int key,pri;        //key：键值，pri：随机的优先级
        int size;           //当前结点为根的子树的结点数量，用于求第k大和rank
}t[M];                  //tree[],存树
```

2.2 初始化与更新

```
void newNode(int x){      //建立只有一个点的树
        cnt++;                //从t[1]开始存储结点，t[0]被放弃
        t[cnt].size = 1;
        t[cnt].ls = t[cnt].rs = 0;  //0表示没有子结点
        t[cnt].key = x;             //key: 键值
        t[cnt].pri = rand();        //pri: 随机产生，优先级
}
void Update(int u){       //更新以u为根的子树的size
        t[u].size = t[t[u].ls].size + t[t[u].rs].size+1;
}
```

三、核心操作原理

3.1 分裂操作 (Split)

功能：把一棵以u为根的Treap树按键值分裂，返回分别以L、R为根的两棵树

- 左树：所有节点的key ≤ x
- 右树：所有节点的key > x

时间复杂度：O(log n)

3.2 合并操作 (Merge)

功能：把树L和树R按优先级合并，合并的隐含前提是L上所有节点的键值都小于R上节点的键值。合并后返回新树的根，显然，新树的根是L和R中优先级最大的那个。

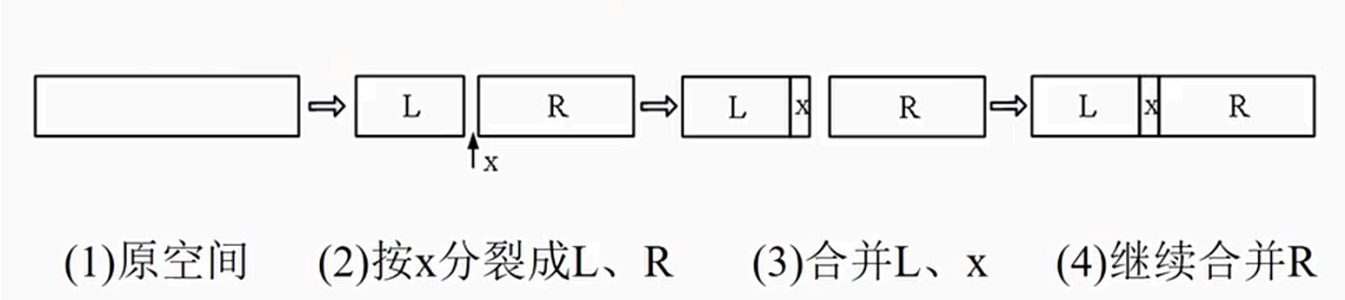
时间复杂度：O(log n)

因为分裂和合并的最多操作次数就是从根到叶子节点，而Treap树的高度的期望值为O(log n)，所以算法的复杂度也为O(log n)。

其他所有的操作:都基于分裂和合并。

四、基本操作实现

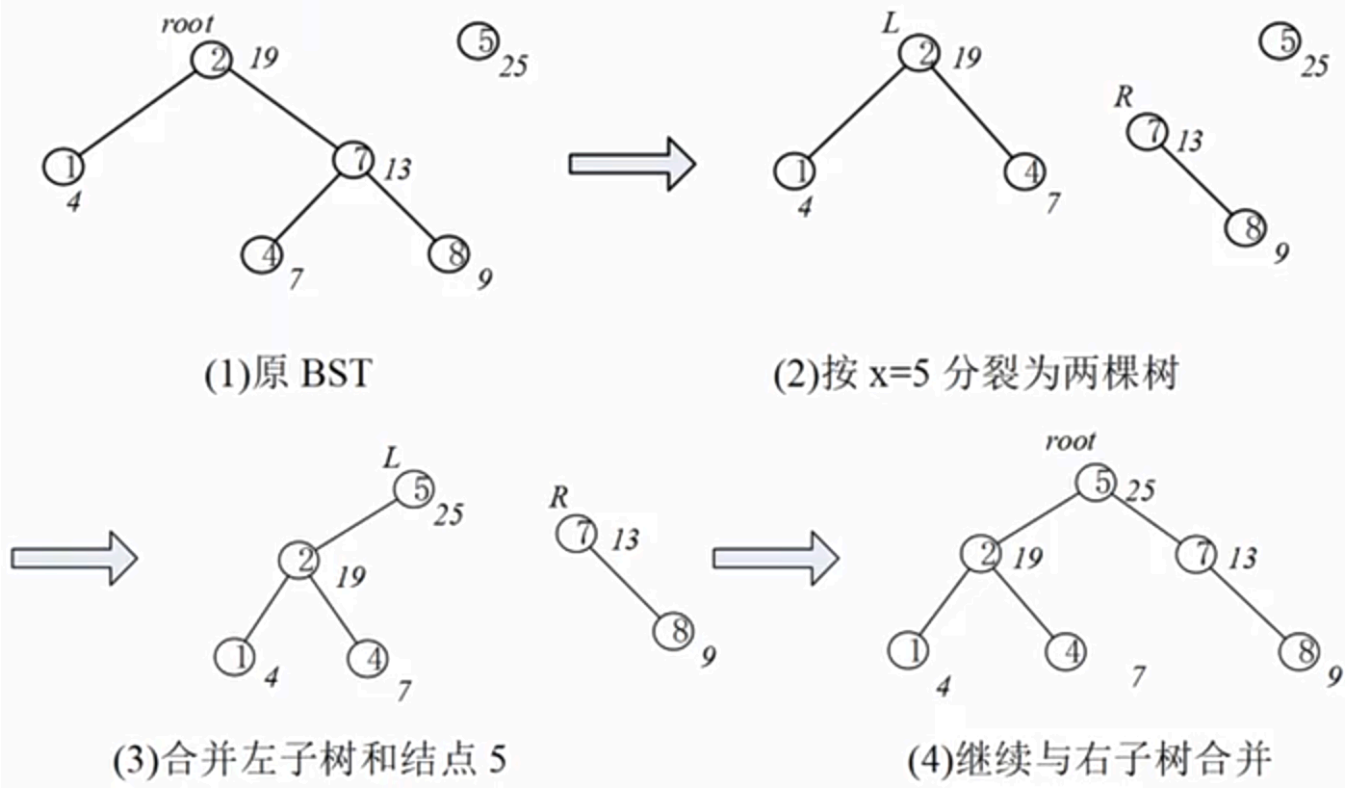
4.1 插入操作



- 图(2):按新结点x的键值把树分裂为L和R两棵
- 图(3):合并L和x
- 图(4):继续与R合并，得到一棵新树

下面举例说明分裂和合并的具体实现，原Treap树包含节点{1，2，4，7，8}，优先级分别为{4，19，7，13，9}，准备加入新节点 x=5。

如(2)所示，把Treap树按节点值x=5分裂成两颗，小于或等于5的节点在左边的树上，大于5的节点在右边的树上。分裂后的两棵树应该仍是Treap树。



如何实现分裂？参见以下代码，分裂只用到了节点的键值key，没有用到节点的优先级，因为分裂的过程不会破坏优先级。

```
void Split(int u,int x,int &L,int &R) {   //权值分裂。返回以L和R为根的2棵树
        if(u == 0){L = R = 0;        return;}      //到达叶子，递归返回
        if(t[u].key <= x){                   //本结点比x小，那么到右子树上找x
                L = u;                           //左树的根是本结点
                Split(t[u].rs, x, t[u].rs, R);   //通过rs传回新的子结点
        }
        else{                                 //本结点比x大，继续到左子树找x
                R = u;                           //右数的根是本结点
                Split(t[u].ls,x,L,t[u].ls);
        }
        Update(u);//更新当前结点的size
}
```

注意，此次的分裂是按键值进行分裂，称为“权值分裂”。分裂后，左树上所有节点的键值key都要小于右树。

下面是合并的代码，合并树L和R。因为L上所有节点键值key都小于R的节点键值，所以合并时只需要考虑节点的优先级pri。合并后的新树，左边是原L的节点，右边是原R的节点。

```
int Merge(int L,int R){                   //合并以L和R为根的两棵树，返回一棵树的根
        if(L==0 || R==0)  return L+R;        //到达叶子。若L==0，就是返回L+R=R
        if(t[L].pri > t[R].pri){             //左树L优先级大于右树R，则L节点是父结点
                t[L].rs = Merge(t[L].rs,R);      //合并R和L的右儿子,并更新L的右儿子
                Update(L);
                return L;                        //合并后的根是L
        }
        else{                                    //合并后R是父结点
                t[R].ls = Merge(L,t[R].ls);      //合并L和R的左儿子,并更新R的左儿子
        Update(R);
                return R;                        //合并后的根是R
        }
}
```

4.2 删除操作

删除一个点x，先通过分裂剥离出x，然后合并。

步骤:

- 把树按x分裂为根小于等于x的树A和大于x的树B;
- 把A分裂为小于x的树C和根等于x的树D;.
- 合并D的左右儿子得E，也就是删除了x;
- 最后合并C、E、B.

注意，以上是“权值分裂”的删除，而“排名分裂”的删除操作略有不同。

4.3 查询排名

求数字x的排名:

- 在每个点上，用size记录以它为根的子树的数量。
- 求数x的排名，把树按x-1分裂成A、B，A中包含了所有小于x的数，
- x的排名等于A的size+1。
- 排名之后合并A、B恢复成原来的树。

4.4 查询第k大

求第k大的数，代码和Treap的“旋转法”一样，不需要分裂和合并操作

4.5 前驱查询

求比x小的数:

- 把树按x-1分裂成A、B
- 在A中找最大的数(利用求第k大数操作)
- 找到后合并A、B恢复成原来的树。

4.6 后继查询

求比x大的数:

- 把树按x分裂成A、B

- 在B中找最小的数(利用求第k大数操作)
- 找到后合并A、B恢复成原来的树。

五、两种分裂方式

5.1 权值分裂（Value Split）

按给定值x分裂，用于普通平衡树操作：

- 左树：key ≤ x
- 右树：key > x

5.2 排名分裂（Rank Split）

按排名k分裂，用于区间操作：

- 左树：前k个元素
- 右树：剩余元素

可以把按权值进行计算的树称为“权值树”，把按排名进行计算的树称为“区间树”。

六、FHQ与Splay对比

在以前的算法竞赛中，Splay树曾经是最常使用的BST树，它的高效率“分裂、合并”功能使得它在众多复杂场合下得到应用。

很多传统的Splay题目现在可以使用FHQ Treap实现，FHQTreap很灵活，效率也很高，代码也更容易写。